télécom saint-étienne
école d'ingénieurs / nouvelles technologies

LABORATOIRE HUBERT CURIEN
UMR • CNRS • 5516 • SAINT-ETIENNE

TIMA

Soutenance de Stage de Fin d'Etudes

03/03/2026 → 19/09/2026

EXTREME LEARNING MACHINE POUR LES RESEAUX DE NEURONES EMBARQUES

Aina Arimanana RAHARISON

Telecom Saint-Etienne

Vendredi 11 Septembre 2026

Encadré par :

Virginie Burnaz FRESSE (LabHC)

Stéphane Mancini (TIMA)

Sommaire

01 Mise en contexte de mon stage

02 Présentation de sujet et objectifs du stage

03 Méthodologie & Resultats

04 Conclusion & Bilan Personnel et Professionnel

1. Mise en contexte de stage

Introduction

Problématique

- Les modèles de réseaux de neurones sont très performants dans l'analyse et la reconnaissance d'images, mais ils nécessitent beaucoup de calcul, de mémoire et d'énergie.

? **Comment les rendre plus légers pour pouvoir les intégrer dans des appareils aux ressources limitées ?**

Solution

Plusieurs pistes de solutions sont possibles :

1. Réduire son empreinte mémoire tout en préservant ses performances
   → au risque d'augmenter son coût de calcul

2. Réduire son cout de calcul tout en préservant les performances
   → au risque d'augmenter son empreinte mémoire

3. Rechercher le meilleur compromis entre les trois : ***performance- Mémoire et calcul***

Contexte de mon stage

Un projet collaboratif

C’est cette recherche d’un équilibre entre **Performance, Mémoire et Calcul** qui a fait naître cette collaboration entre deux laboratoires aux expertises complémentaires.

LABORATOIRE HUBERT CURIEN
UMR • CNRS • 5516 • SAINT-ETIENNE

TiMA

Concevoir une méthode frugale pour déployer des modèles neuronaux aussi bien dans le cloud que sur des appareils autonomes.

Virginie Burnaz FRESSE
Equipe : Image Science et Computer Vision

- *Optique, Photonique & Surface*
- *IA, Sécurité & Image*

Développer des modèles d’IA suffisamment légers et efficaces pour être exécutés au plus près du capteur

Stéphane MANCINI
Equipe : Circuits, Devices and System Integration

- *Sécurité & Fiabilité matérielle*
- *Capteurs , Images Processing*
- *AI For Design et Test*

2. Présentation du sujet et objectifs du stage

Travaux antérieurs

2021 - 2024 : Thèse de Florent CROZET (TIMA, Directeur de thèse : Stéphane Mancini)

Méthode : Network Random Compression (ou NRC) [1]

- Réduire le nombre de poids à stocker

  → Stocker seulement une partie des poids du modèle entraîné et on remplace les autres.

- Remplacer certains des poids conservés par des valeurs aléatoires

  → Au lieu de mémoriser les poids conservés, leurs valeurs sont regénérés à la volée par un générateur de nombre aléatoire, pendant l'utilisation du modèle.

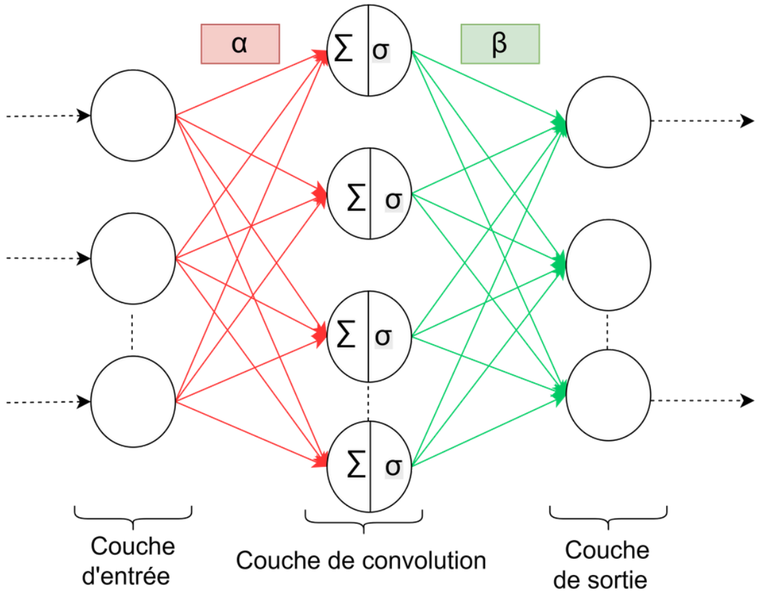
Objectifs de NRC

- Réduction de l'empreinte mémoire
- Coût de calcul moins coûteux

[1] F. Crozet, « Extreme Learning Machine for embedded neural networks », PHD thèse, Université Grenoble Alpes, 2024.

Principe de l'adaptation de ELM vers un NRC

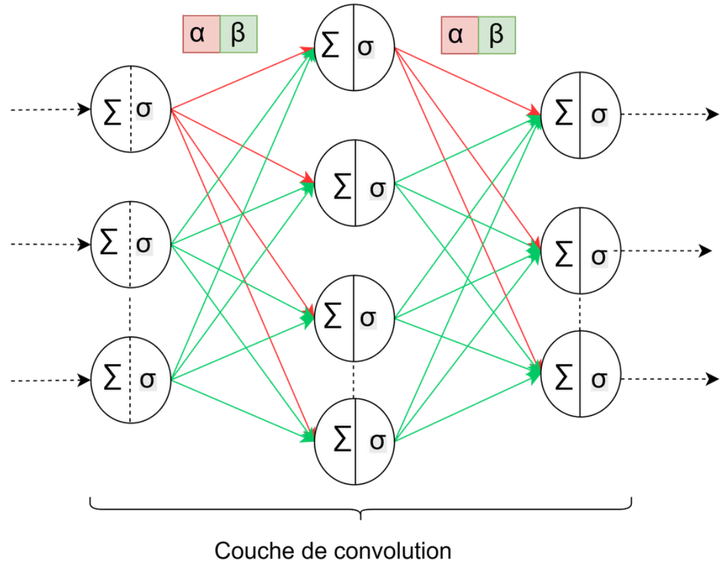
Extreme Learning Machine [2]



α : poids aléatoires

β : poids entraînés

NRC



[2] G.-B. Huang, Q.-Y. Zhu, et C.-K. Siew, « Extreme learning machine: Theory and applications », Neurocomputing, vol. 70, no 1-3, p. 489-501, déc. 2006, doi: 10.1016/j.neucom.2005.12.126.

Missions et objectifs du stage

Mission principale et objectifs

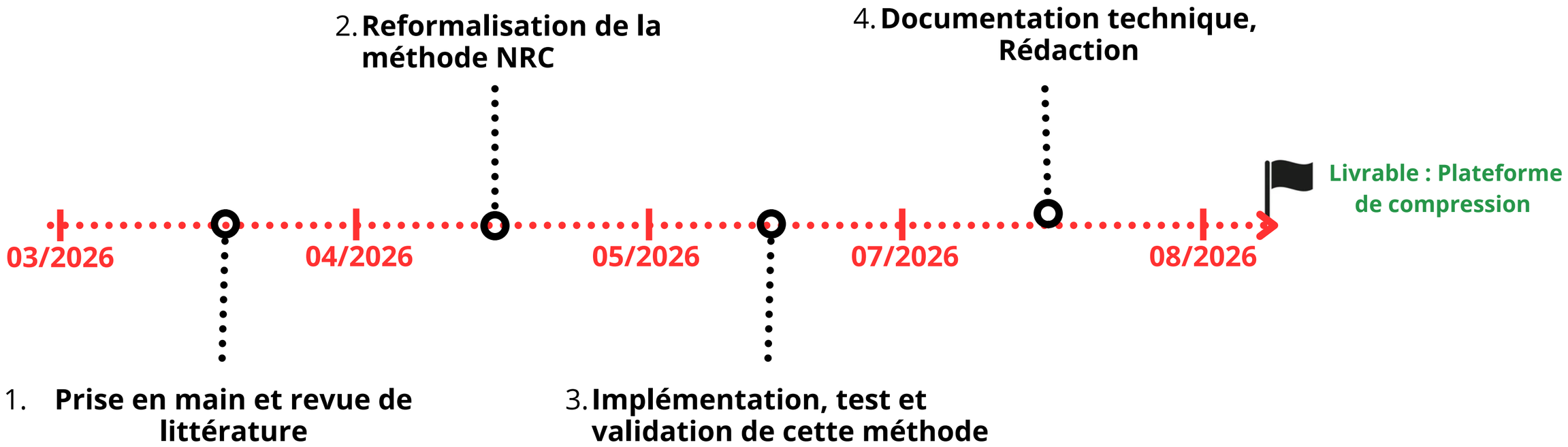
Mission principale :
Développer une plateforme de compression des réseaux de neurones basés sur cette méthode NRC

But: Valider expérimentalement cette méthodologie de compression des réseaux de neurones NRC, proposée par Florent CROZET [1]

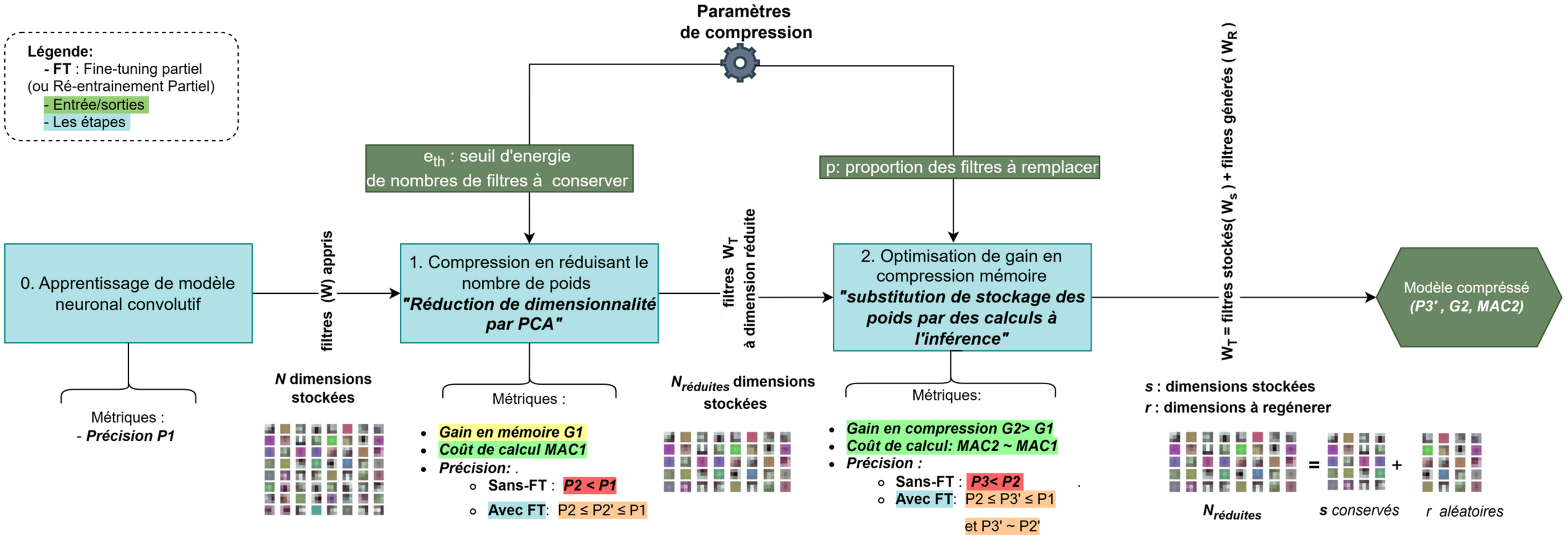
3. Méthodologie et Résultats

Méthodologie

Organisation des missions



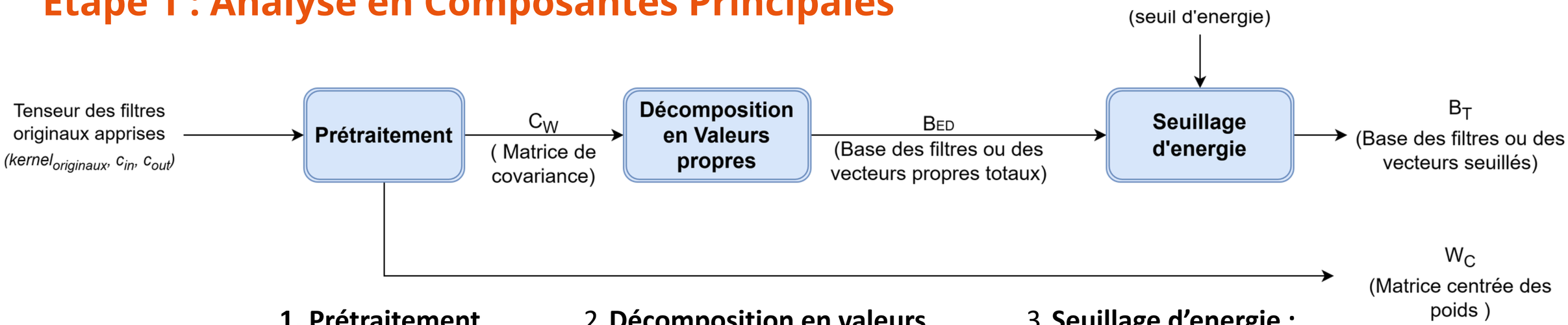
Pipeline de la méthode NRC



Filtres = *matrice des poids*
PCA *: Principal Component Analysis (Analyse en Composantes Principales)*
MAC*: Multiply and ACcumulate (Multiplieur - Additionneur)*

Pipeline de la méthode NRC

Etape 1 : Analyse en Composantes Principales



1. Prétraitement

- *Aplatissement* : $W \in M_{d,cout}$ et $d=k_H \cdot k_W \cdot c_{in}$
- *Centrage :* $W_C=W-w_{mean}$

2. **Décomposition en valeurs propres** : Soit : $C_{W=} W_C . W_C^T \in R^{d \times d}$

- $C_W=PDP^T$ tel que $P=B_{ED} \in R^{d \times d}$ et $D \in R^{d \times d}$

D: matrice diagonale des valeurs propres

3. **Seuillage d'energie :**

- **Sous-espace de projection :**
  - $B_T=B_{ED}(:s) \in R^{d \times t}$

t<d : nombre de vecteurs propres conservés

- Définition du ratio d'énergie

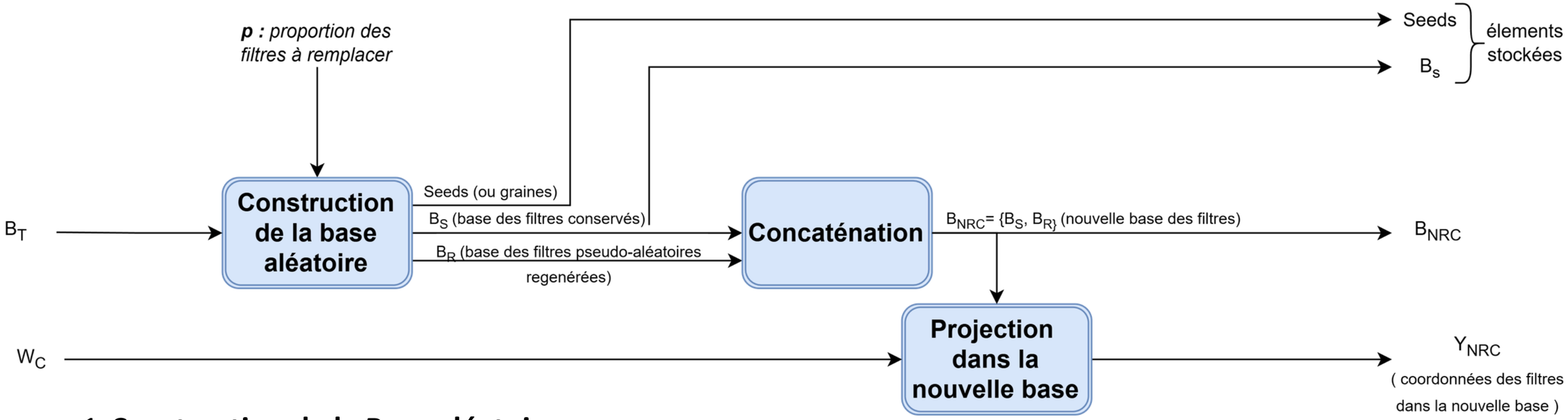
$$e = (\text{trace}(D_{[0,t]} \div \text{trace}(D) \times 100$$

- Seuillage d'énergie : **e≥eTh**

Reconstruction complète des poids : $\hat{W} = \hat{W}_C + w_{mean} = B_T Y_T + w_{mean}$

Pipeline de la méthode NRC

Etape 2 : Remplacement des filtres par des filtres pseudo-aléatoires



1. **Construction de la Base aléatoire**
   - Calcul des dimensions
     r=⌊t×p⌋ à générer
   - Sélection de B_S :
     s=t - r et $B_s = B_T[:s]$
   - Génération de B_R à l'aide :
     - *Distance de Grassmann*
     - *LFSR*

2. **Concaténation**

$$B_{NRC} = \{B_S , B_R\}$$

3. **Projection**

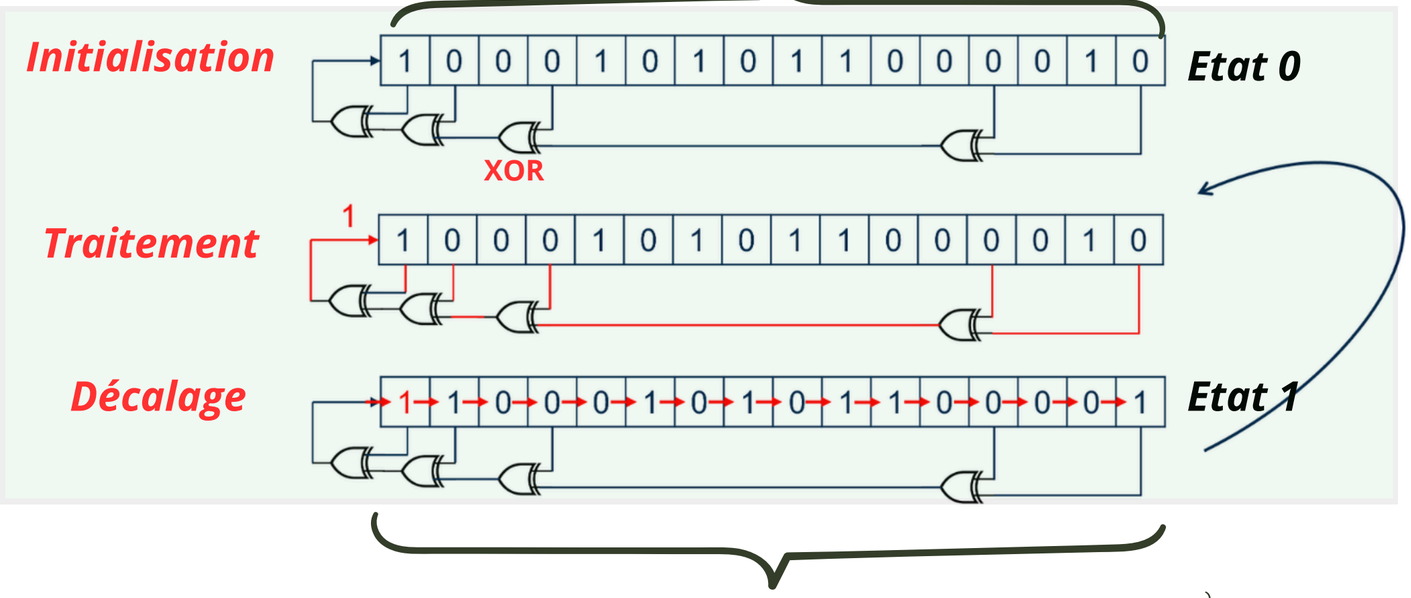
$$Y_{NRC} = B_{NRC}^T . W_C$$

Pipeline de la méthode NRC

Sous-étape de 2 : Construction de la base hybride $B_{NRC}=\{B_s \mid B_R\}$

Tirage des K graines candidates

LFSR



$(valeur_1, valeur_2, \ldots\ldots\ldots\ldots\ldots, valeur_d)$

Vecteur aléatoire candidat de dimension d rajouté dans la base aléatoire B_R

Calcul de distance de Grassmann

$d_G(B_T, B_{NRC}\ [Vs|V_R(v_candidat)])$

argmin (dist)

Meilleur seed (graine) retenu

(int32)

(1ère itération de remplacement)

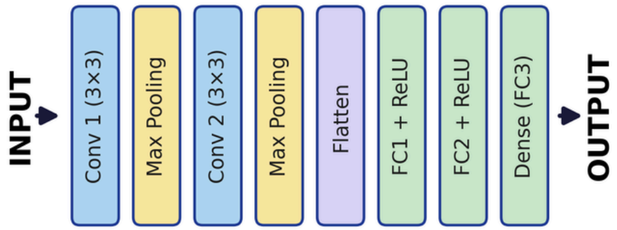
Répétion *r* fois de ce processus

(r itérations de remplacement)

Tests & Résultats

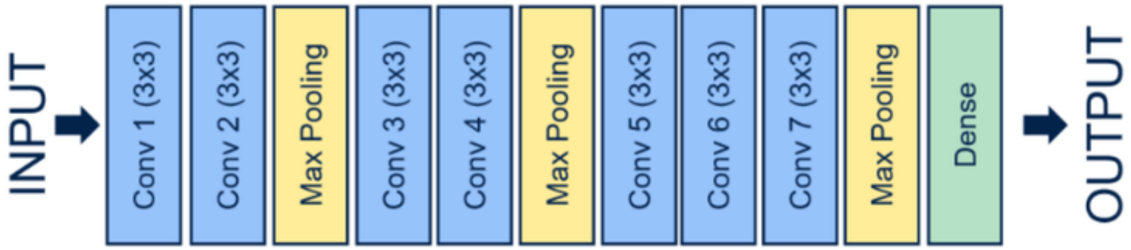
Cadre expérimental

LeNet (*P1=58,7% sur 4 epochs*)



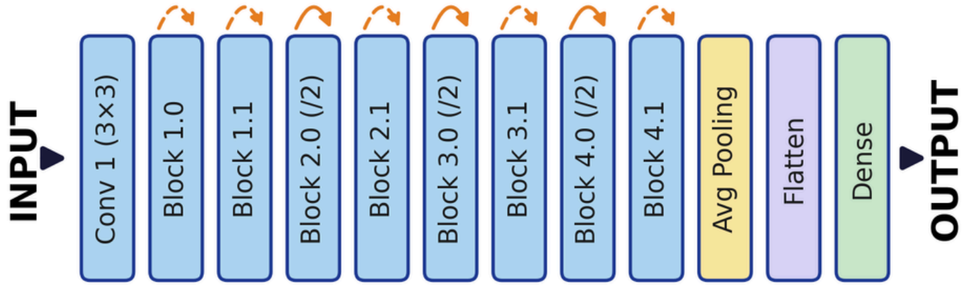
Paramètre de convolution = 1048 paramètres

VGG16 *(P1=75,4% sur 4 epochs)*



Paramètre de convolution = ~7 M de paramètres

ResNet18 *(P1= 80,8% sur 4 epochs)*



Bloc résiduel = 2× Conv 3×3 + BN + ReLU. Skip connection : - - - identité —— projection (Conv 1×1 + BN, stride 2)

Paramètre de convolution = ~11,1 M de paramètres

e_{th} ={25%, 50%, 75%, 90%} **&** ***p = {0%, 25%, 50%, 75%}***

(seuil d'energie PCA) *(proportion de remplacement des filtres)*

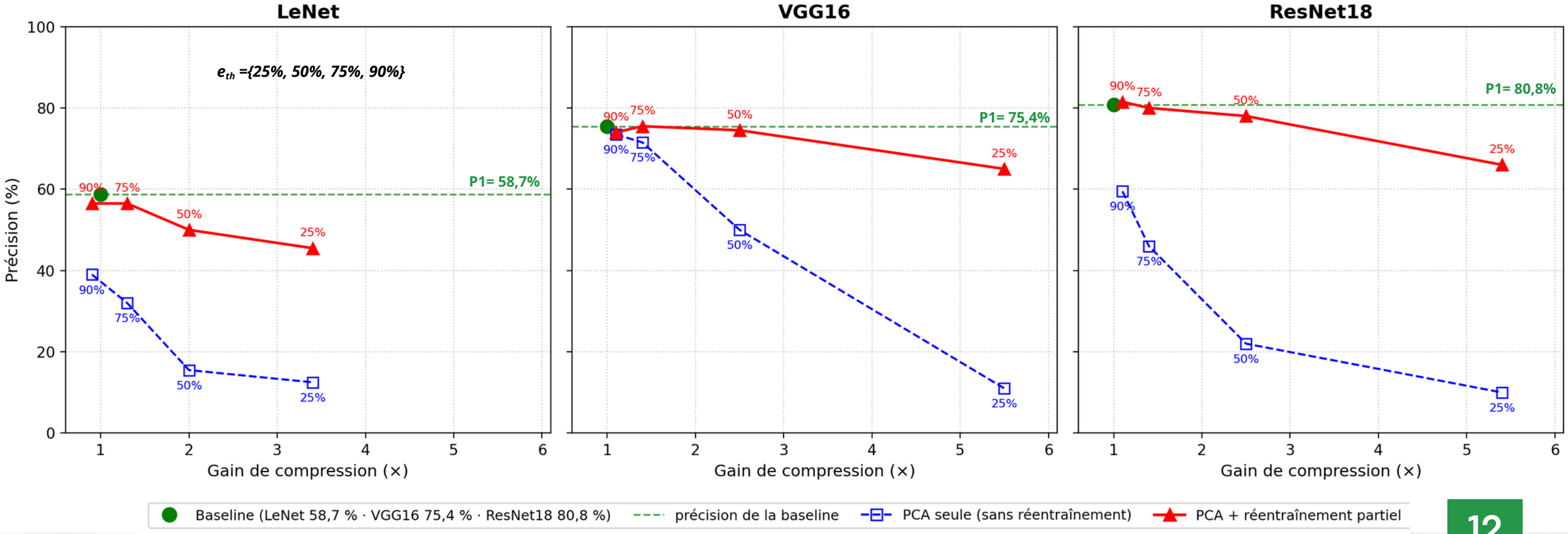
(Précision & Gain en mémoire)
(Précision & Nombre de MAC)
(Nombre de MAC & Gain en mémoire)

Test & Resultats à la sortie de PCA

Précision et gain de compression à la sortie de PCA (Etape 1)

Objectifs :

- Etudier le compromis entre précision et compression de mémoire

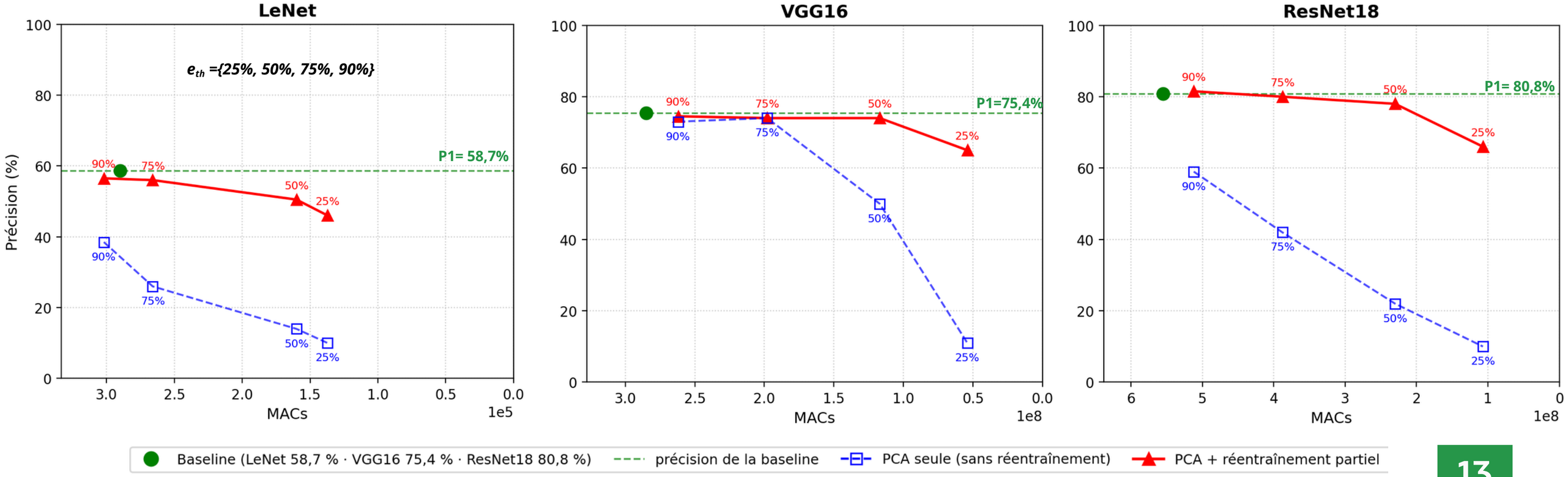


Test & Resultats à la sortie de PCA

Précision et nombre de MAC à la sortie de PCA (Etape 1)

Objectifs :

- Etudier le compromis entre la Précision et le nombre de MACs



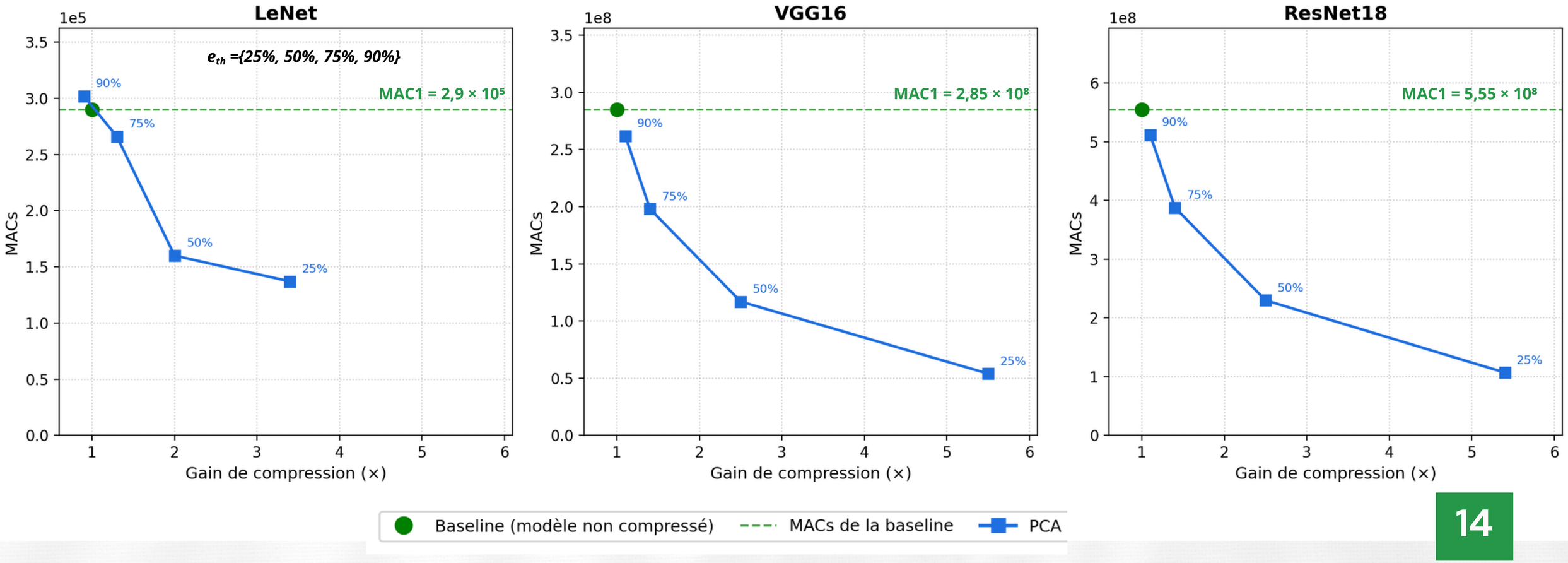
MAC: Multiply and ACcumulate (Multiplieur - Additionneur)

Test & Resultats à la sortie de PCA

Evolution de MAC et gain de compression à la sortie de PCA (Etape 1)

Objectifs :

- Etude du compromis entre mémoire et le nombre de MAC à la sortie de PCA

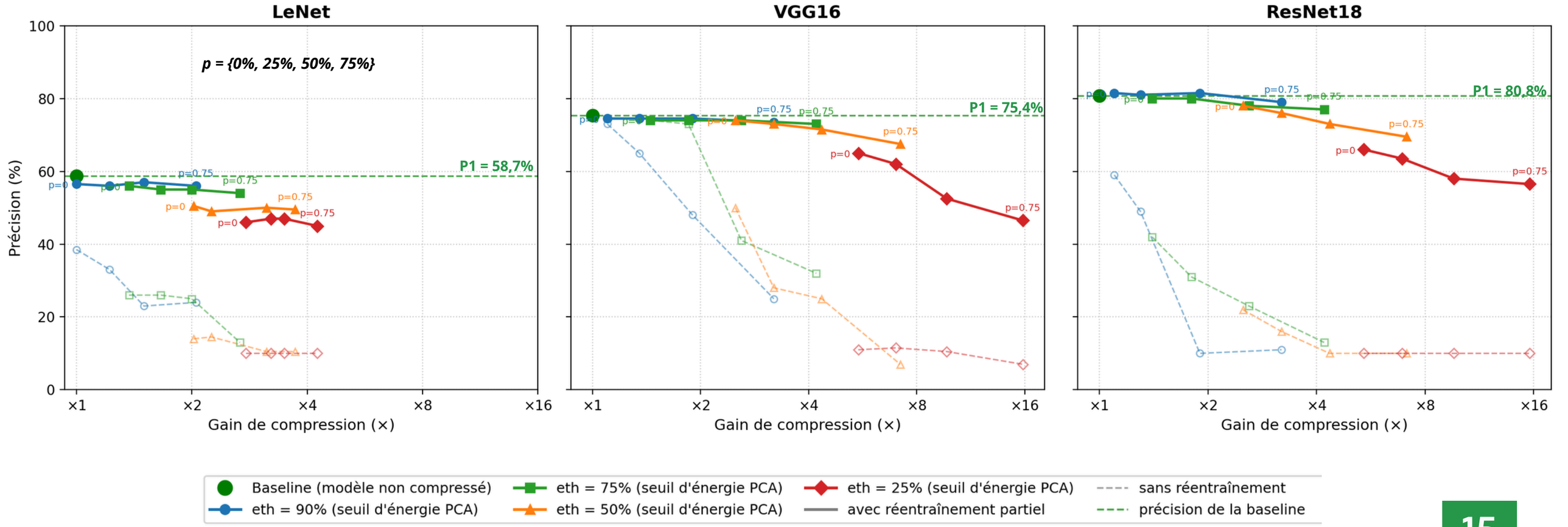


Test & Resultats à la sortie de NRC

Précision et gain de compression à la sortie de NRC (Etape 1 et 2)

Objectifs :

- Etudier le compromis entre précision et l'optimisation du gain en mémoire

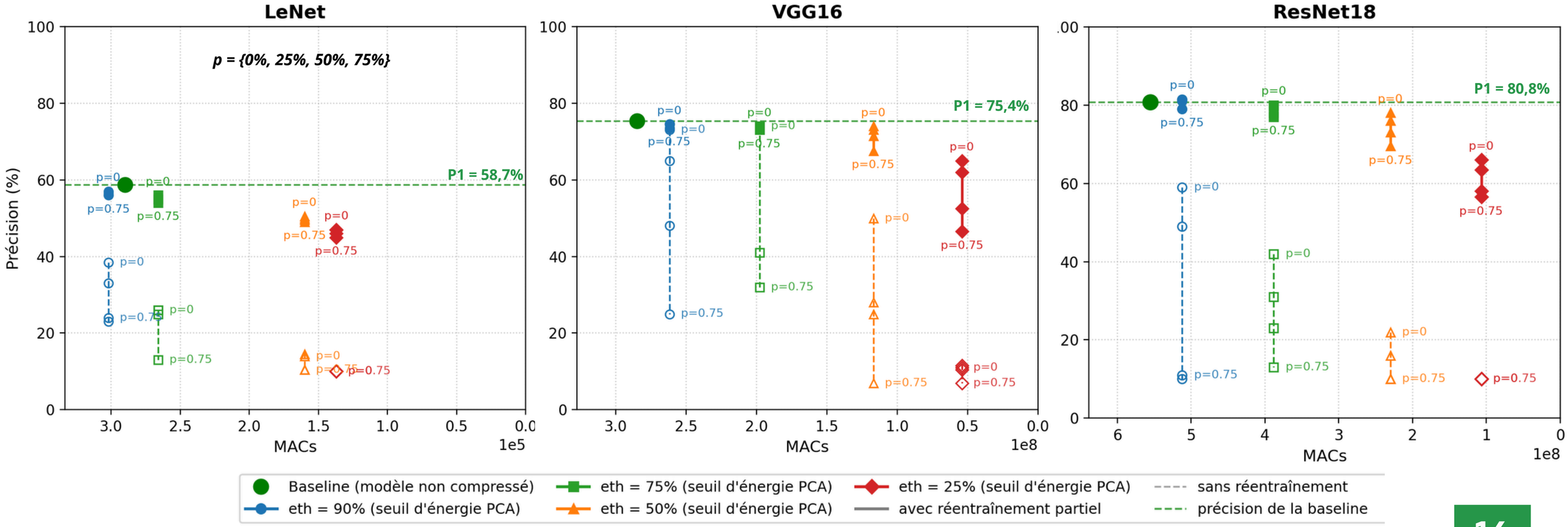


Test & Resultats à la sortie de NRC

Précision et le nombre de MAC à la sortie de NRC (Etape 1 et 2)

Objectifs :

- L'impact du remplacement des filtres sur le nombre de MAC

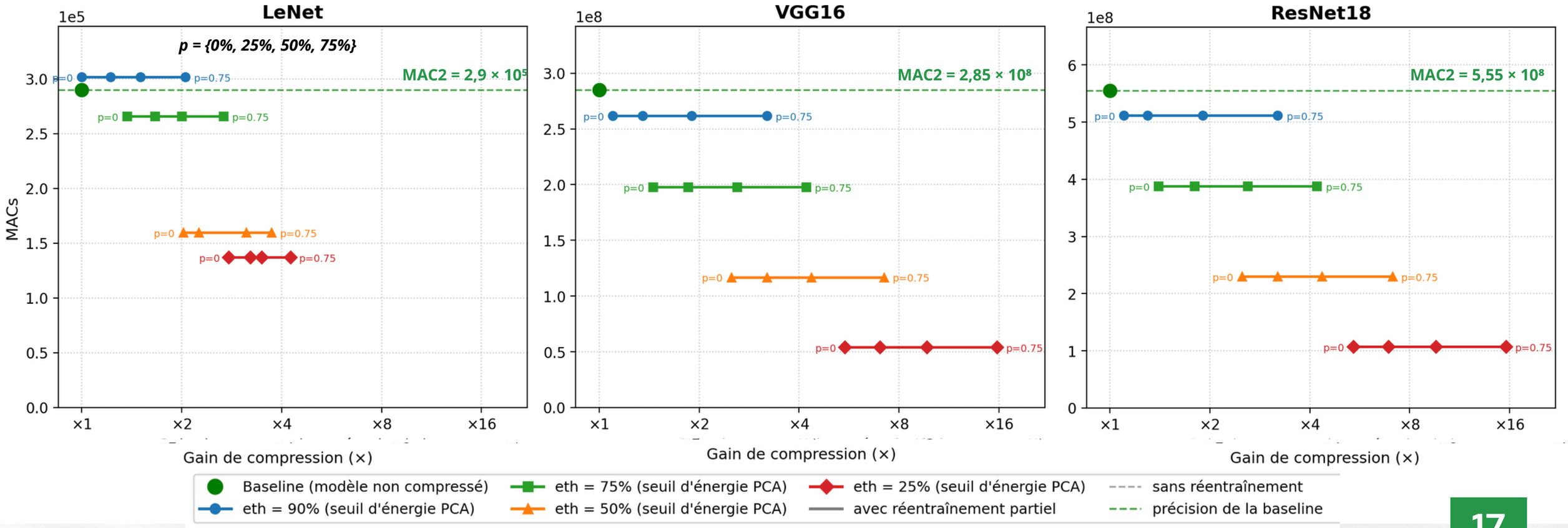


Test & Resultats à la sortie de NRC

Evolution de MAC et gain en compression à la sortie de NRC (Etape 1 et 2)

Objectifs :

- Etude de l'impact de remplacement des filtres sur le nombre de MAC et le compromis entre mémoire et nombre de MAC



Présentation de la plateforme

```
options:
  --model {lenet,resnet18,vgg_mini}
  --eth FLOAT [FLOAT ...]
  --p FLOAT [FLOAT ...]
  --retrain-after-pca
  --retrain-after-pseudo
  --epochs-initial EPOCHS_INITIAL
                        (default: 4)
  --epochs-retrain EPOCHS_RETRAIN
                        (default: 2)
  --lr LR                Learning rate
(Adam)  (default: 0.001)
  --batch-size BATCH_SIZE
  --save-models
  --save-dir SAVE_DIR    default:
checkpoints)
  --device DEVICE
  --seed SEED     Graine LFSR (default:
42)
  --K K             Candidats par
vecteur LFSR (default: 100)
  --output OUTPUT    (défaut :
résultats_YYYYMMDD_HHMMSS.xlsx)
  --output-dir OUTPUT_DIR
                         (default:
results)
```

Script Python

run.py

balayage eTh × p
(+ baseline)

results/résultats_20260430_144647.xlsx

Sans_reentrainement | Avec_reentrainement_PCA | Avec_reentrainement_Pseudo

Modèle	eTh	p	Accuracy (%)	Conv params	Mém. conv (KB)	Gain compr.	Seeds (nb)	Seeds (o)	s_layers
lenet	90%	0.00	71.20	118 476	462.80	2.48	0	0	[6, 12]
lenet	75%	0.25	70.05	83 640	326.72	3.51	7	28	[4, 9]
lenet	50%	0.50	66.80	51 020	199.30	5.75	14	56	[3, 6]
lenet	25%	0.75	61.40	29 310	114.49	10.01	21	84	[2, 4]
réf. (non compr.)	–		72.10	293 640	1147.03	1.00	0	0	–

Un classeur .xlsx par exécution

- 3 feuilles : Sans_reentrainement, Avec_reentrainement_PCA, Avec_reentrainement_Pseudo
- 1 ligne par couple (eTh, p) + 1 ligne de référence · cellules colorées automatiquement
- run --baseline produit en plus results/baseline_AAAAMMJJ_HHMMSS.xlsx

checkpoints/

lenet_eth75%_p0.00_ft_state.pth
lenet_eth75%_p0.00_ft_manifest.json
lenet_eth75%_p0.00_ft_manifest_arrays.npz
lenet_eth50%_p0.25_ft_state.pth
lenet_eth50%_p0.25_ft_manifest.json
lenet_eth50%_p0.25_ft_manifest_arrays.npz
… (1 triplet par couple eTh × p, et par variante : _ft, _pca_ft)

1 triplet par (eTh, p)

- **.pth** – state_dict PyTorch : poids fine-tunés (coeff. β, BN…)
- **.json** – métadonnées + graines PRNG (eth, p, s, n_keep, n_rand, seeds)
- **.npz** – tableaux numpy : BNRC_keep (n_keep × d), w_mean

Modèle compressé sauvegardé pour chaque configuration

Nom : <modèle>_eth<seuil>%_p<ratio>[_pca]_ft_{state.pth | manifest.json | manifest_arrays.npz}
.pth + .json + .npz suffisent à recharger et rejouer exactement le réseau NRC

4. Conclusion & Bilan Personnel et Professionnel

Conclusion

Résumé

- Développement de la plateforme de compression basé sur la méthode NRC
- Test et validation de la plateforme sur 3 architectures de neurones (LeNet, VGG16 et ResNet18)

Apports concrets

- Plateforme de compression réutilisable et extensible
  - Documentation technique plus accessible
  - Restructuration du code facilitant sa mise en oeuvre

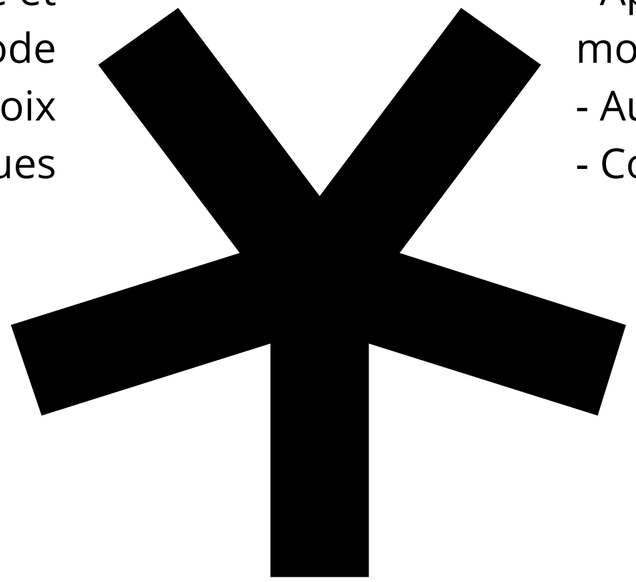
Suite du projet:

- Tester la plateforme sur d’autres architectures de modèles neuronaux
- Extension de la méthode NRC en ajoutant la quantification
- Intégration de la méthode dans un système embarqué

Bilan personnel et projet professionnel

1. **Atteinte des objectifs :**
- Adaptation rapide au contexte et enjeu de la méthode
- Regard critique sur des choix techniques

2. **Savoir-faire et savoir-être développés:**
- Approfondir mes compétences en compression des modèles neuronales
- Autonomie de gestion de projet et rigueur méthodologique
- Communication scientifique (retour critique)



3. **Outils maîtrisés :**
- Outils de recherche et gestion bibliographique *(Zotero, Undermind)*
- Framework IA *(Pytorch, les bibliothèques Python (scikit_learn, numpy))*
- Environnement de développement *(Visual Studio Code)*

5. **Projet à court et moyen terme:**
- Recherche de mon premier poste
- Faire de la R&D dans le domaine de l'IA Frugale

4. **Evolution de mes choix et intérêts :**
- Stage en vision par ordinateur **(SIP1)**
- Stage en Traitement d'Images moléculaires **(SIP2)**
- Stage en IA Frugale **(SFE)**
- *Ingénieur de Recherche et Développement d'une IA plus éthique et responsable*
- ***Secteurs : automobile, aéronautique, agriculture, santé***

Remerciements

Tuteurs entreprise :

- Virginie Burnaz FRESSE
- Stéphane MANCINI

Tutrice académique :

- Elena KACHAN

Mes collègues de Laboraotire :

- Carl
- Corentin
- Julien
- Lucie
- Philippe

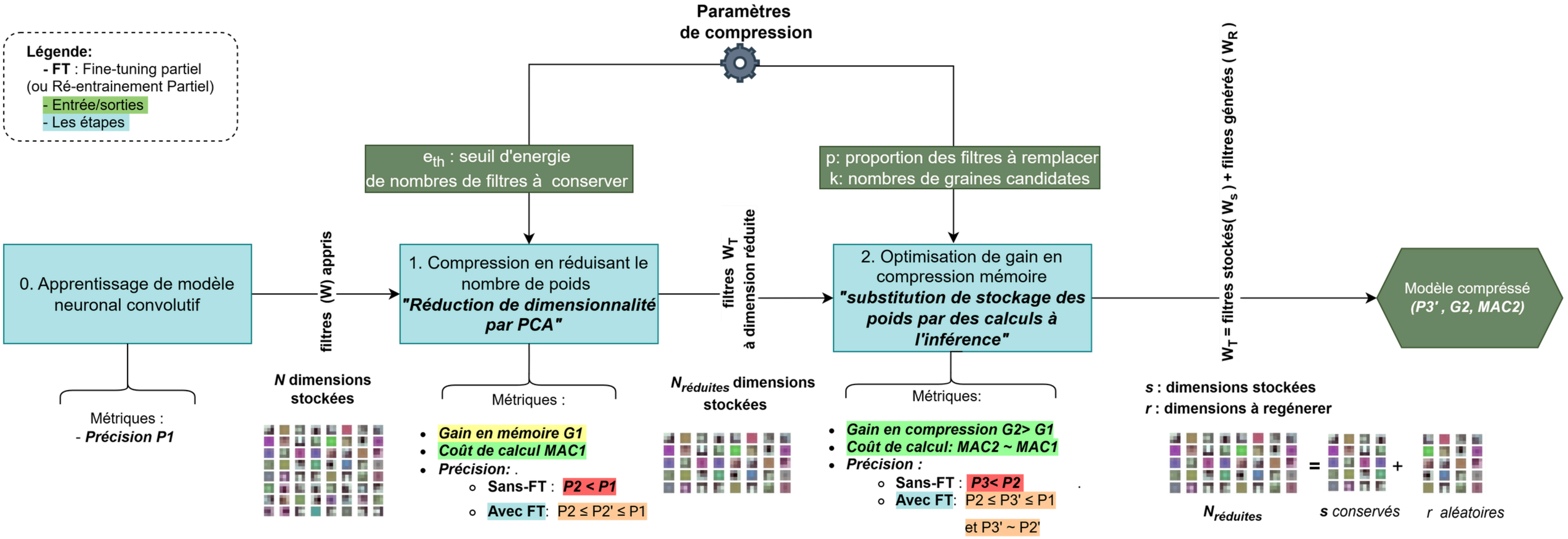
Aina Arimananana RAHARISON

Merci
pour votre attention

IV. Annexe

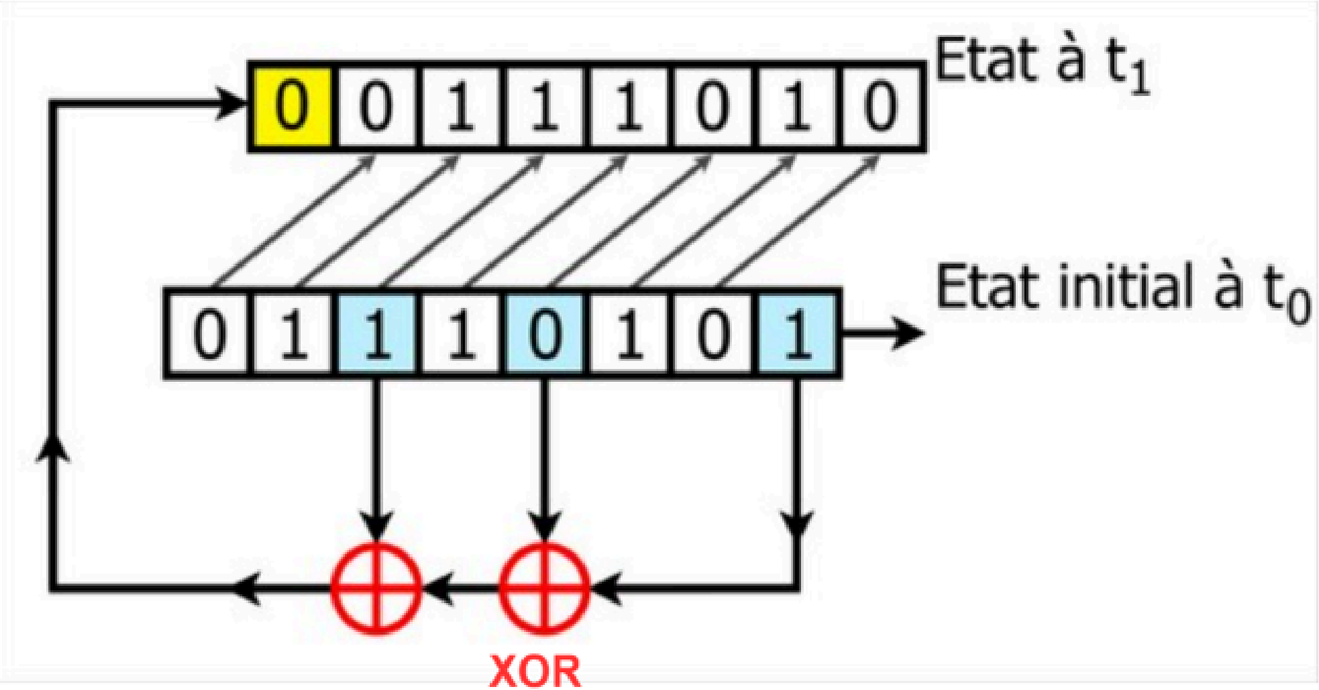
Annexe

Pipeline complet de la méthode



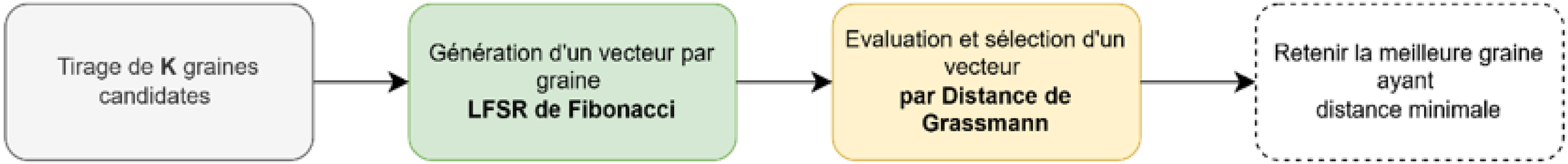
Annexe

Fonctionnement du générateur de nombre pseudo aléatoire LFSR



Annexe

Etape de construction de filtre pseudo-aléatoire



Tirage des K graines candidates :

- Pour chaque vecteur destiné à remplacer un vecteur de base appris, K graines (ou seeds) numériques (ex : 16 bits) sont tirées aléatoirement.
- Chacune de ces graines constitue un candidat potentiel pour initialiser la génération du vecteur pseudo-aléatoire correspondant.

Génération des vecteurs aléatoires :

- Puis pour chaque graine (ou seed) candidate, un vecteur est généré à l'aide d'un générateur de nombre pseudo-aléatoires (ex : dans notre cas un LFSR de Fibonacci (registre à décalage à rétroaction linéaire).
- Cette graine, à la différence du vecteur lui-même, sera conservée en mémoire, en revanche le vecteur aléatoire complet sera régénéré à la volée à l'inférence par une l'exécution du même LFSR avec la même graine retenue optimale

Evaluation et sélection :

- Les K vecteurs générés ne sont pas tous équivalent du point de vue de la précision : un vecteur pseudo-aléatoire est optimale et retenue quand il permet à la base de couvrir le même sous-espace vectoriel que la base optimale issue de l'ACP.

Annexe

run.py

balayage eTh × p
(+ baseline)

results/résultats_20260430_144647.xlsx

Sans_reentrainement | Avec_reentrainement_PCA | Avec_reentrainement_Pseudo

Modèle	eTh	p	Accuracy (%)	Conv params	Mém. conv (KB)	Gain compr.	Seeds (nb)	Seeds (o)	s_layers
lenet	90%	0.00	71.20	118 476	462.80	2.48	0	0	[6, 12]
lenet	75%	0.25	70.05	83 640	326.72	3.51	7	28	[4, 9]
lenet	50%	0.50	66.80	51 020	199.30	5.75	14	56	[3, 6]
lenet	25%	0.75	61.40	29 310	114.49	10.01	21	84	[2, 4]
réf. (non compr.)	–		72.10	293 640	1147.03	1.00	0	0	–

Un classeur .xlsx par exécution

- 3 feuilles : Sans_reentrainement, Avec_reentrainement_PCA, Avec_reentrainement_Pseudo
- 1 ligne par couple (eTh, p) + 1 ligne de référence · cellules colorées automatiquement
- run --baseline produit en plus results/baseline_AAAAMMJJ_HHMMSS.xlsx

checkpoints/

```
lenet_eth75%_p0.00_ft_state.pth
lenet_eth75%_p0.00_ft_manifest.json
lenet_eth75%_p0.00_ft_manifest_arrays.npz
lenet_eth50%_p0.25_ft_state.pth
lenet_eth50%_p0.25_ft_manifest.json
lenet_eth50%_p0.25_ft_manifest_arrays.npz
```

1 triplet par (eTh, p)

- **.pth** — state_dict PyTorch : poids fine-tunés (coeff. β, BN...)
- **.json** — métadonnées + graines PRNG (eth, p, s, n_keep, n_rand, seed
- **.npz** — tableaux numpy : BNRC_keep (n_keep × d), w_mean